AI Impact Claims API

Technical Report

COMP3011 Web Services and Web Data Coursework

Student name: _____ Student ID: _____

Date: March 2026

1. Project overview

The project is a data-driven REST API that records structured claims about AI's impact on jobs, creativity, and the environment. The implementation combines a CRUD resource (claims) with a small SQL-backed analytics resource (AI exposure scores) so that the system satisfies the module's minimum technical requirements while remaining distinctive and closely aligned with the dissertation theme.

The submitted prototype exposes six HTTP endpoints through FastAPI: POST /claims, GET /claims, GET /claims/{claim_id}, PATCH /claims/{claim_id}, DELETE /claims/{claim_id}, and GET /analytics/high-exposure. All responses are JSON. SQLite is used as the persistence layer and SQLAlchemy provides the ORM and session management.

2. Requirements mapping

Brief requirement	Implementation choice	Evidence in project
CRUD model linked to a database	claims table persisted in SQLite via SQLAlchemy	POST/GET/PATCH/DELETE /claims
At least four endpoints	Six endpoints provided	Swagger UI and API documentation
JSON responses and status codes	FastAPI response models and HTTPException handling	201, 200, 204, 404, 422 shown in docs
Data-driven design	Imported occupation-exposure sample data	GET /analytics/high-exposure
Oral exam deliverables	README, API docs PDF, technical report, slides	submission package

3. Technology stack and rationale

FastAPI was selected because it provides typed request validation, automatic OpenAPI schema generation, and interactive Swagger documentation with almost no configuration. That makes it especially suitable for a coursework project where the API itself, its documentation, and the oral demonstration all need to stay tightly aligned.

SQLite was selected as the SQL database for the submitted version because it is lightweight, self-contained, and easy to run locally without additional server setup. This reduced deployment risk under the short deadline while still satisfying the brief's requirement for a relational database. SQLite is also appropriate for a prototype with a modest write volume and simplified operational constraints.

SQLAlchemy was used to separate persistence logic from request handling and to keep the codebase extensible. Although a raw sqlite3 implementation would have been shorter, the ORM structure is easier to justify architecturally and would support later migration to PostgreSQL if the project were scaled up.

4. Architecture and data model

The application follows a straightforward three-layer structure. FastAPI routes handle HTTP requests and response models. SQLAlchemy models define the database schema. The SessionLocal dependency manages database sessions per request. This separation keeps request validation, persistence, and API documentation consistent.

Table	Purpose	Key fields
claims	Primary CRUD entity for user-authored AI impact statements	id, title, category, description, source_url, occupation_code, created_at

ai_exposure_scores	Imported analytics dataset used for ranking occupations	occupation_code, occupation_title, exposure_score

The design intentionally keeps the relational model small. A future iteration would normalise occupations into a separate table and enforce a foreign-key link from claims. For the submitted version, occupation_code remains optional so that claims can be recorded even when the user has no matching occupation code.

5. Dataset use and analytics

To make the project data-driven, a CSV import script loads a small occupation-exposure dataset into ai_exposure_scores. The current repository uses a reduced sample for demonstration, but the structure is compatible with a fuller AI Occupational Exposure (AIOE) dataset. The analytics endpoint GET /analytics/high-exposure filters by minimum score and returns the top rows ordered by exposure_score descending.

This endpoint demonstrates SQL filtering, ordering, and limit controls in a way that is easy to explain during the oral exam. It also strengthens the connection to the dissertation topic by moving the API beyond purely manual CRUD and towards evidence-backed exploration.

6. Error handling and testing

The submitted prototype relies on FastAPI's built-in validation and SQLAlchemy queries. Successful create operations return 201, read/update operations return 200, deletes return 204, missing records return 404, and invalid request bodies return 422 validation errors.

Testing was primarily carried out through Swagger UI and curl from the local terminal. This manual approach was sufficient to validate CRUD operations, database persistence, and the analytics route under time pressure. A production-ready extension would add automated pytest integration tests for endpoint behaviour and import scripts.

7. Challenges and lessons learned

The main implementation challenge was reducing project scope without losing the dissertation theme. The original idea was an observatory combining jobs, creativity, and environmental indicators, but the deadline made a smaller vertical slice more realistic. The final design therefore prioritised one robust CRUD model plus one analytics table. This decision improved reliability and made the oral demo easier to control.

A second practical challenge was path and environment handling during the CSV import step. Because the script was run from a subdirectory, Python initially failed to resolve project-level imports. This was solved by appending the project root to sys.path in the import script. This reinforced the importance of environment awareness and reproducibility.

8. Limitations and future work

The current version has several clear limitations. First, the analytics dataset is only a small prototype subset. Second, the system has no authentication, so all claims are effectively public and unauthenticated. Third, the database is SQLite rather than a networked production database. Fourth, the analytics layer currently provides only one ranking endpoint rather than richer longitudinal comparisons.

Future work would therefore include: (1) replacing the sample dataset with a full published occupation-exposure source, (2) adding an occupations table and foreign keys, (3) implementing authentication and rate-limiting, (4) migrating to PostgreSQL for stronger concurrency and deployment flexibility, and (5) adding trend endpoints that compare claim frequencies or occupation exposure over time.

9. Generative AI declaration

Generative AI was used throughout the project in a declared, methodical way. It was used for idea scoping, endpoint planning, debugging guidance, drafting the import workflow, and improving the presentation/report structure. AI suggestions were treated as starting points rather than final answers. All final code was manually reviewed, adapted to the project requirements, and tested locally through FastAPI and curl.

The most useful AI contributions were: narrowing the project into a manageable vertical slice, proposing the claims data model, suggesting example response bodies and status codes, and diagnosing the local server and import-path issues encountered during implementation. This aligns with the brief's expectation that higher marks come from thoughtful and creative use of GenAI rather than undeclared or purely arbitrary usage.

References

- University of Leeds. COMP3011 Coursework 1 Brief (2025/26).
- FastAPI documentation: automatic OpenAPI and interactive docs.
- SQLite documentation: self-contained SQL database engine.
- Felten, Raj, and Seamans (2021) AI Occupational Exposure dataset and related GitHub repository.

GitHub repository (fill in before submission): https://github.com/<your-username>/<your-repo>